

Rede Social Gamificada & Sustentável

Domain Driven Design Using Java — Sprint 4

Equipe: RankUp | **Solução:** RankUp+

Tiago Borges Dos Santos	**RM 569926**
Flavio Luiz Kuratomi Junior	**RM 571211**
João Victor de Jesus Bernardo	**RM 568729**
Pedro Andreotti Pugliesi	**RM 569357**
Henrique Osuka	**RM 571324**

Sumário

1. Objetivo e Escopo do Projeto
2. Principais Funcionalidades
3. Arquitetura e Tecnologias
4. Tabela de Endpoints (API RESTful)
5. Protótipo - Telas Implementadas
6. Modelo Entidade-Relacionamento (MER)
7. Diagrama de Classes
8. Deploy e Acesso
9. Procedimentos para Rodar a Aplicação

1. Objetivo e Escopo do Projeto

O RankUp+ é uma rede social gamificada e sustentável, desenvolvida no contexto do Challenge SoulUp. A solução incentiva práticas sustentáveis do dia a dia: o usuário registra ações como o uso de transporte público, a reciclagem de lixo e a economia de energia, acumula pontos, calcula o CO2 que deixou de ser emitido, sobe de nível (Iniciante, Bronze, Prata e Ouro) e compete em um ranking com os demais usuários.

Nesta Sprint, o projeto evoluiu da aplicação de console da Sprint 3 para uma API RESTful em Java, integrada ao front-end em React e publicada em um servidor de aplicação (Render). A API expõe os recursos da solução em formato JSON, mantém os dados em um banco de dados Oracle e aplica as regras de negócio em uma camada própria, separada do acesso a dados.

2. Principais Funcionalidades

- Cadastro de usuários, com opção de conta Administrador por meio de um código informado no cadastro.
- Login por e-mail, com dois perfis de acesso: Comum e Administrador.
- Registro de ações sustentáveis, com cálculo automático de pontos e do CO2 não emitido.
- Perfil do usuário com pontuação total, nível, CO2 acumulado e histórico de ações.
- Cálculo de nível a partir da pontuação e da diversidade de ações registradas.
- Ranking geral (top 10) e consulta da posição do próprio usuário.
- Atualização dos dados do usuário.
- Administração (somente Administrador): listar todos os usuários e remover usuários, com o histórico junto.

3. Arquitetura e Tecnologias

A aplicação é organizada em camadas, cada uma com uma responsabilidade:

Camada	Pacote	Responsabilidade
Resource (Controller)	controller	Recebe as requisições HTTP, chama a camada de negócio e devolve JSON com o status correto.
BO (Service)	service	Validações e regras de negócio (cadastro, login, registro de ação, ranking, permissões de admin).
DAO	dao	Acesso ao banco de dados Oracle (JDBC), com a ConnectionFactory para a conexão.
Model	models	Classes de domínio que representam as entidades e o banco de dados.
DTO e Mapper	dto, mapper	Objetos de entrada e saída da API e a conversão entre modelo e DTO.
Exceções	exception	ApiException e o tratador central de erros, que devolve respostas no formato { "erro": "mensagem" }.
Configuração	config	CorsFilter: libera o acesso do front-end à API (CORS) apenas para localhost e para domínios *.vercel.app.

Item	Tecnologia
Linguagem	Java 21
Framework	Spring Boot 4.1.1 (Spring Web)
Build	Maven
Banco de dados	Oracle (driver ojdbc17, acesso via JDBC)
Container e deploy	Docker e Render
Front-end	React, desenvolvido na disciplina de Front-End Design Engineering

O framework utilizado foi o Spring Boot, conforme alinhado com a professora da disciplina.

4. Tabela de Endpoints (API RESTful)

Todos os endpoints recebem e devolvem JSON. Os endpoints marcados como Admin exigem o cabeçalho X-Usuario-Id, com o id de um usuário do tipo Administrador.

Método	URI	Descrição	Acesso	Sucesso	Erros
POST	/usuarios	Cadastra um usuário (código de admin opcional)	Público	201	400, 409
POST	/login	Entra com o e-mail cadastrado	Público	200	400, 404
GET	/usuarios/{id}	Perfil: pontuação, nível e CO2 evitado	Público	200	404
PUT	/usuarios/{id}	Atualiza nome, e-mail e idade	Público	200	400, 404, 409
GET	/acoes	Lista as ações sustentáveis disponíveis	Público	200	-
POST	/usuarios/{id}/registros	Registra uma ação (soma pontos e atualiza o nível)	Público	201	400, 404
GET	/usuarios/{id}/registros	Histórico de ações do usuário	Público	200	404
GET	/ranking?limite=10	Ranking geral (top N, padrão 10)	Público	200	400
GET	/usuarios/{id}/posicao	Posição do usuário no ranking	Público	200	404
GET	/usuarios	Lista todos os usuários	Admin	200	403
DELETE	/usuarios/{id}	Remove o usuário e o histórico dele	Admin	204	403, 404

Códigos de status utilizados

Código	Significado	Quando ocorre
200	OK	Consulta ou atualização realizada com sucesso.
201	Created	Usuário ou registro de ação criado.
204	No Content	Usuário removido com sucesso (sem corpo na resposta).
400	Bad Request	Campo obrigatório ausente ou inválido (e-mail, idade, quantidade, data).
403	Forbidden	Cabeçalho X-Usuario-Id ausente ou usuário que não é Administrador.
404	Not Found	Usuário, e-mail ou ação inexistente.
409	Conflict	E-mail já cadastrado.
500	Internal Server Error	Falha inesperada no servidor (por exemplo, banco de dados indisponível).

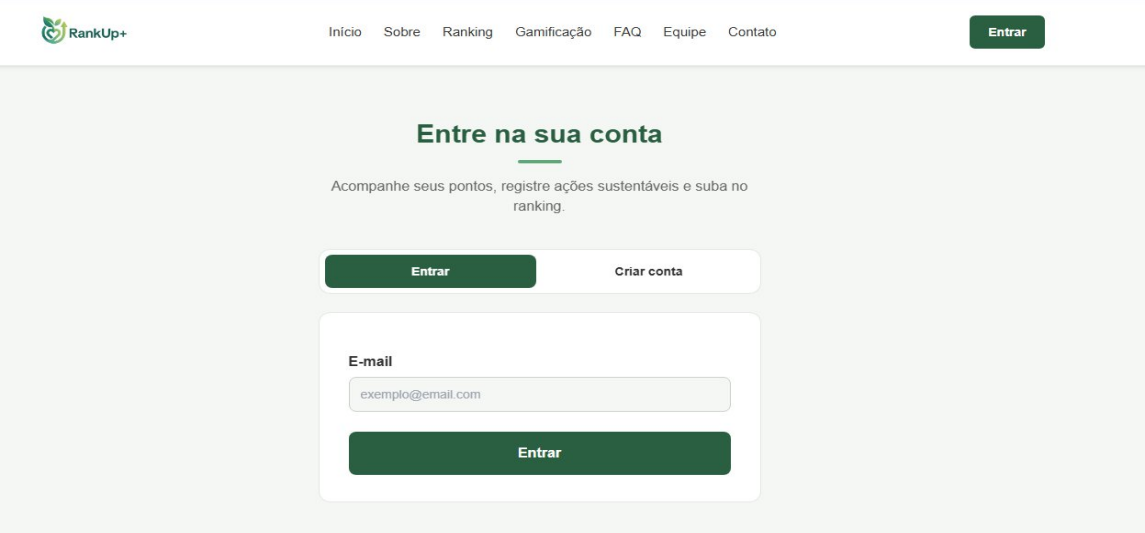
As respostas de erro seguem sempre o formato:

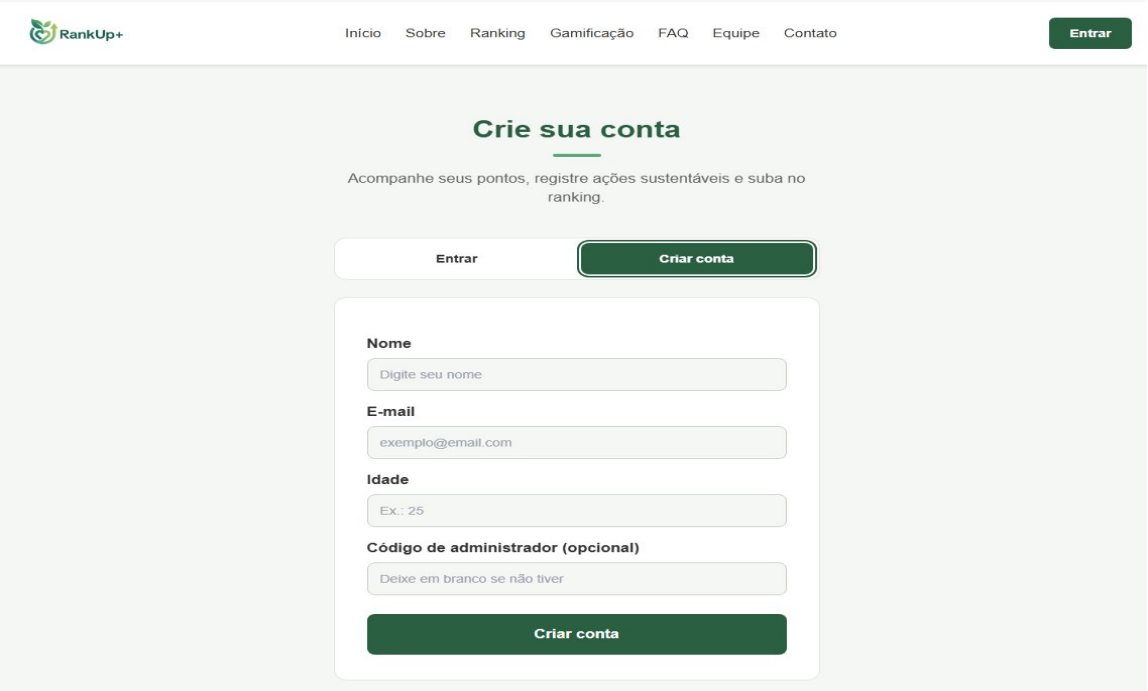
```
{ "erro": "mensagem explicando o problema" }
```

5. Protótipo — Telas Implementadas

Capturas de tela reais do sistema em funcionamento, com uma explicação resumida de cada tela.

5.1 Cadastro e Login

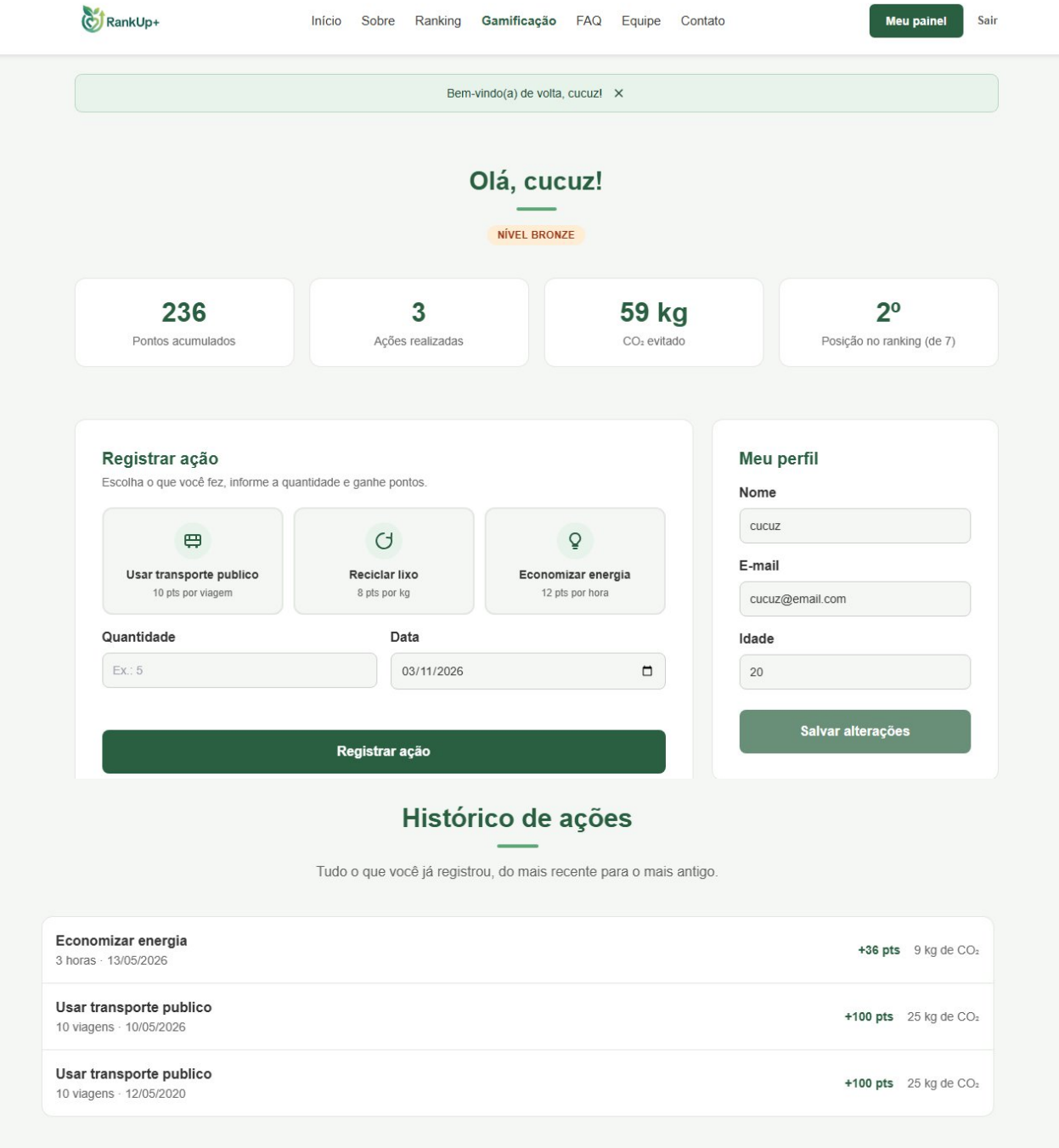




Explicação: Primeira tela do sistema. O usuário alterna entre Criar conta e Entrar. No cadastro informa nome, e-mail e idade; o campo de código de administrador é opcional e, se estiver correto, a conta é criada como Administrador. No login, informa apenas o e-mail.

Endpoints utilizados: POST /usuarios e POST /login.

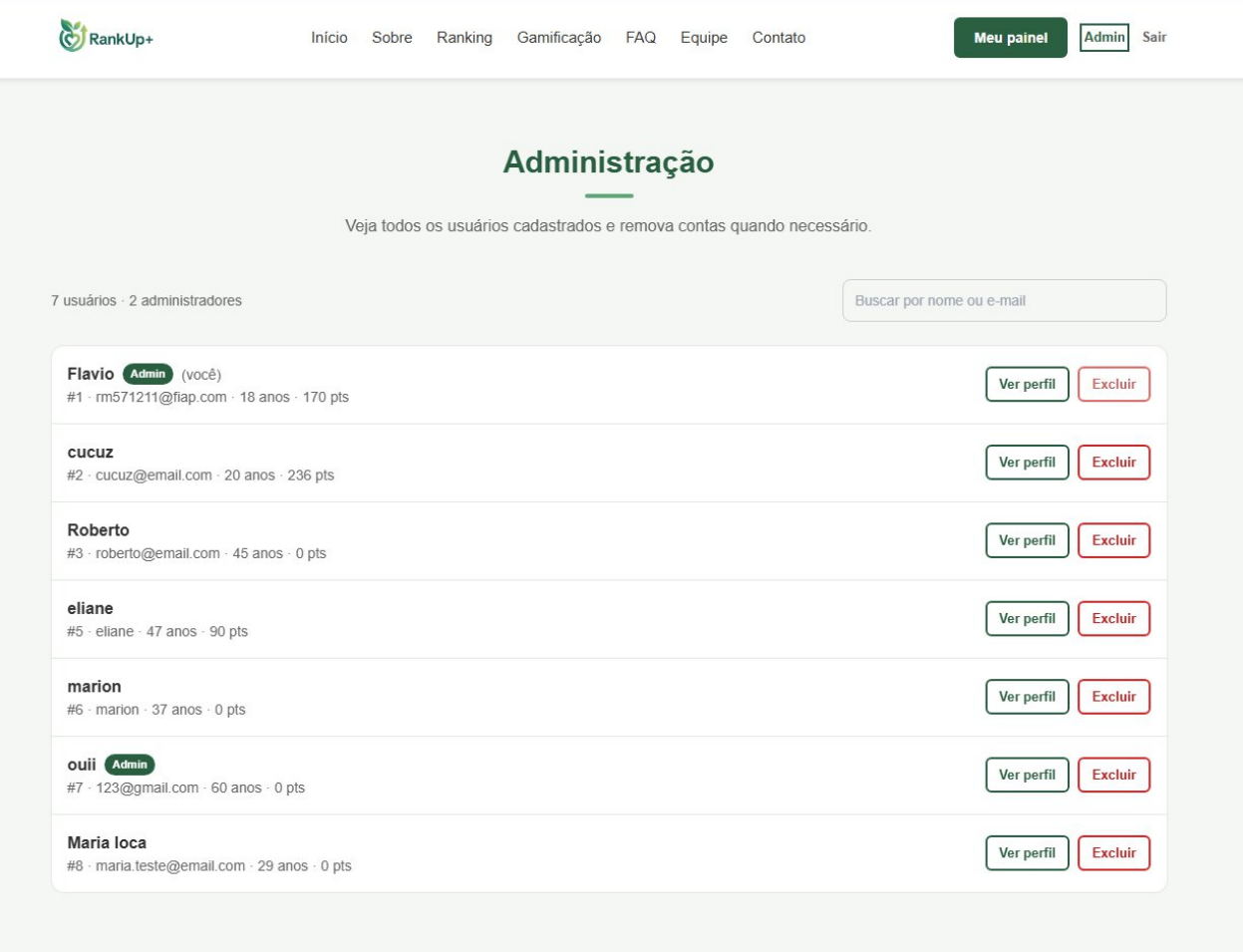
5.2 Menu do Usuário Comum



Explicação: Mostra a saudação, a pontuação total e o nível atual, além dos atalhos para registrar ação, ver perfil, consultar a posição no ranking e abrir o ranking geral.

Endpoints utilizados: GET /usuarios/{id}.

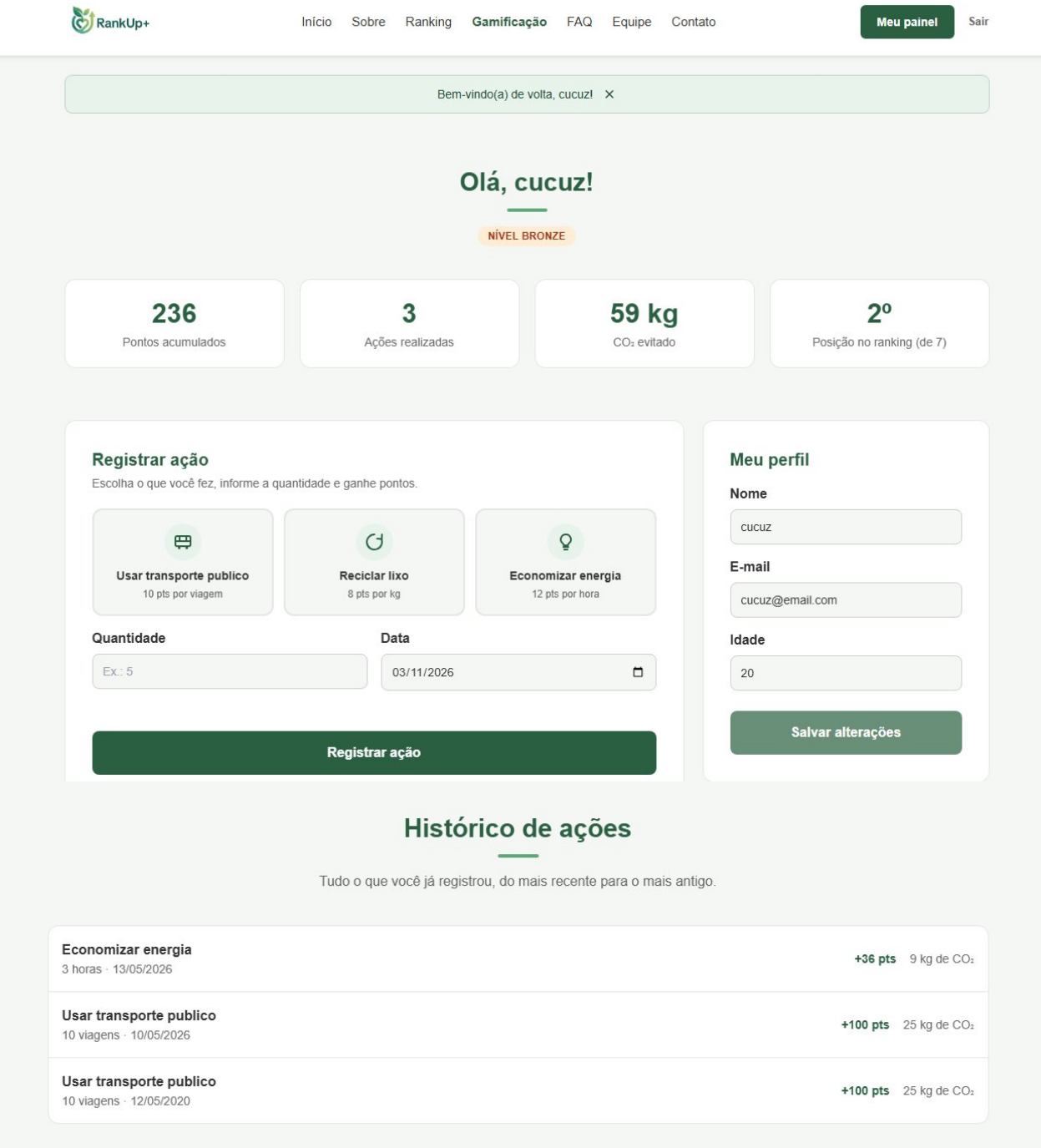
5.3 Menu do Administrador



Explicação: Igual ao menu do usuário Comum, com uma seção extra de Administração para listar todos os usuários e remover usuários.

Endpoints utilizados: GET /usuarios e DELETE /usuarios/{id} (cabeçalho X-Usuario-Id).

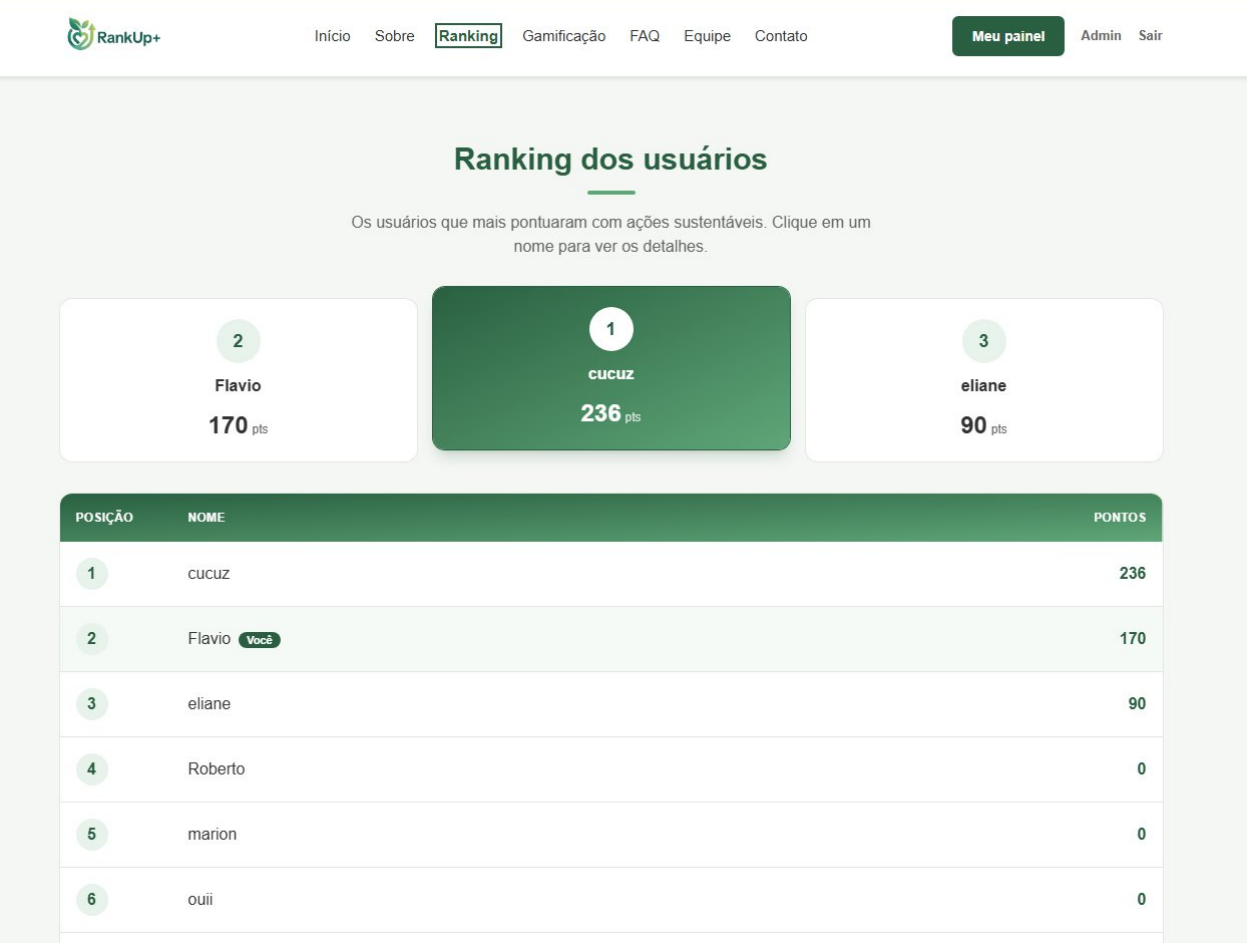
5.4 Perfil e Pontuação



Explicação: Exibe os dados do usuário, o nível, os pontos totais, a quantidade de ações registradas, o CO2 evitado e o histórico de ações com data, quantidade e pontos.

Endpoints utilizados: GET /usuarios/{id} e GET /usuarios/{id}/registros.

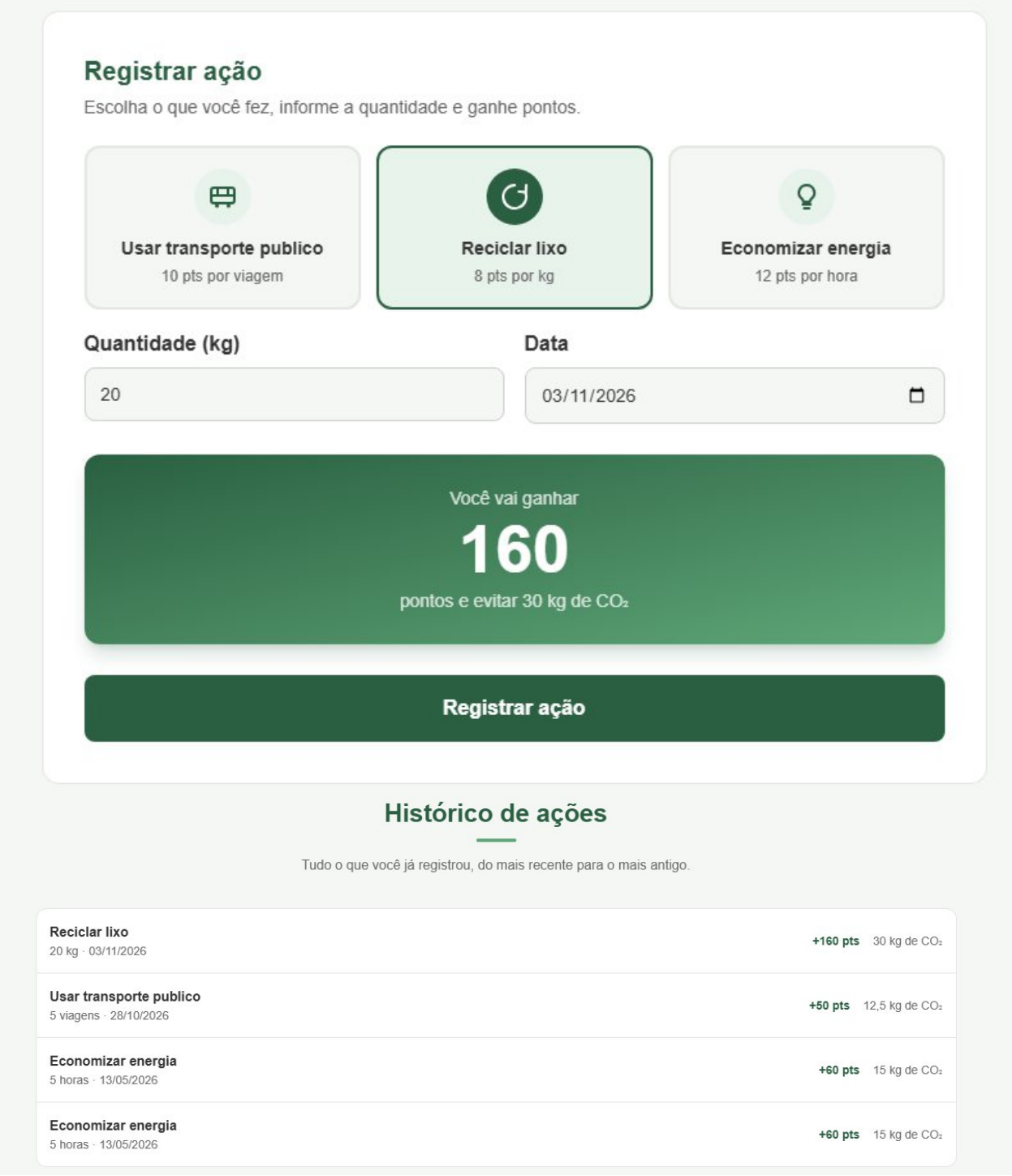
5.5 Ranking

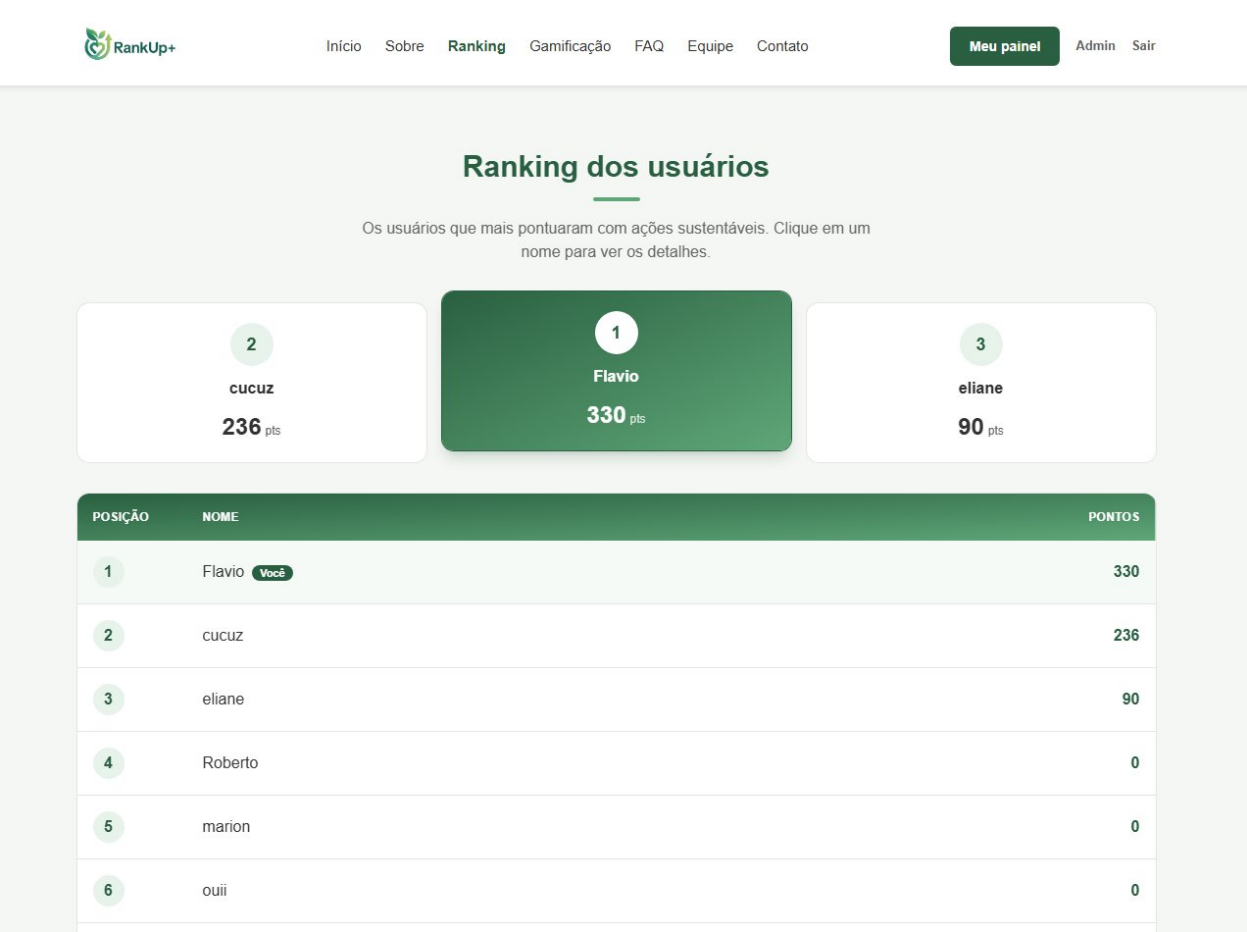


Explicação: Lista os 10 usuários com mais pontos, com destaque para os três primeiros. A linha do usuário logado é destacada e um cartão no rodapé informa a posição dele entre todos os usuários.

Endpoints utilizados: GET /ranking e GET /usuarios/{id}/posicao.

5.6 Registrar Ação Sustentável



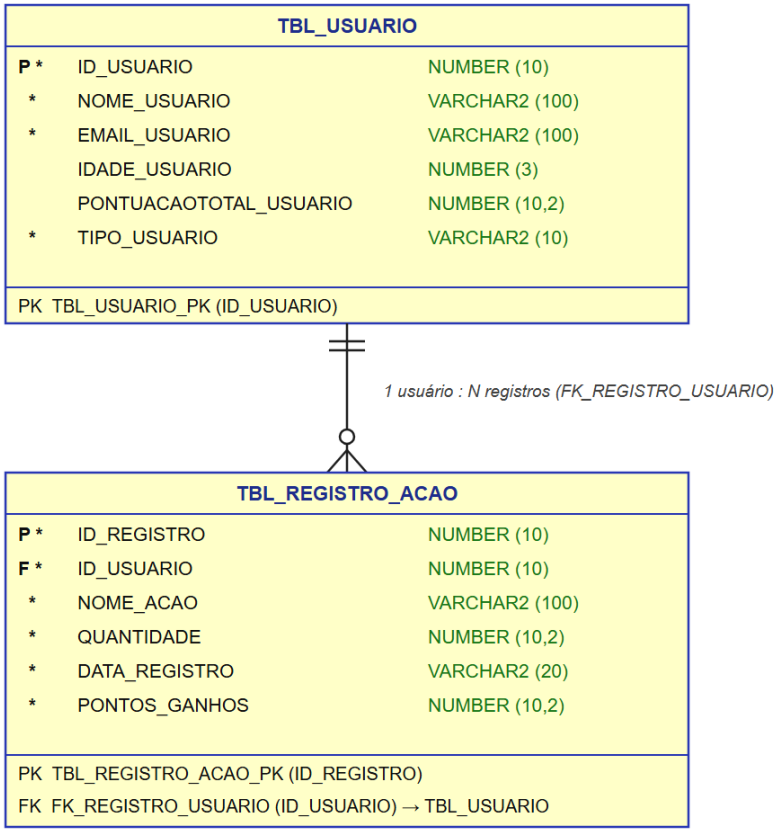


Explicação: O usuário escolhe o tipo de ação, informa a quantidade (o rótulo muda conforme a ação) e a data. Ao confirmar, o sistema calcula os pontos, atualiza a pontuação e o nível e salva o registro no histórico.

Endpoints utilizados: GET /acoes e POST /usuarios/{id}/registros.

6. Modelo Entidade-Relacionamento (MER)

O banco de dados possui duas tabelas, relacionadas por uma chave estrangeira: um usuário pode ter vários registros de ação, e cada registro pertence a exatamente um usuário.



6.1 Estrutura das Tabelas

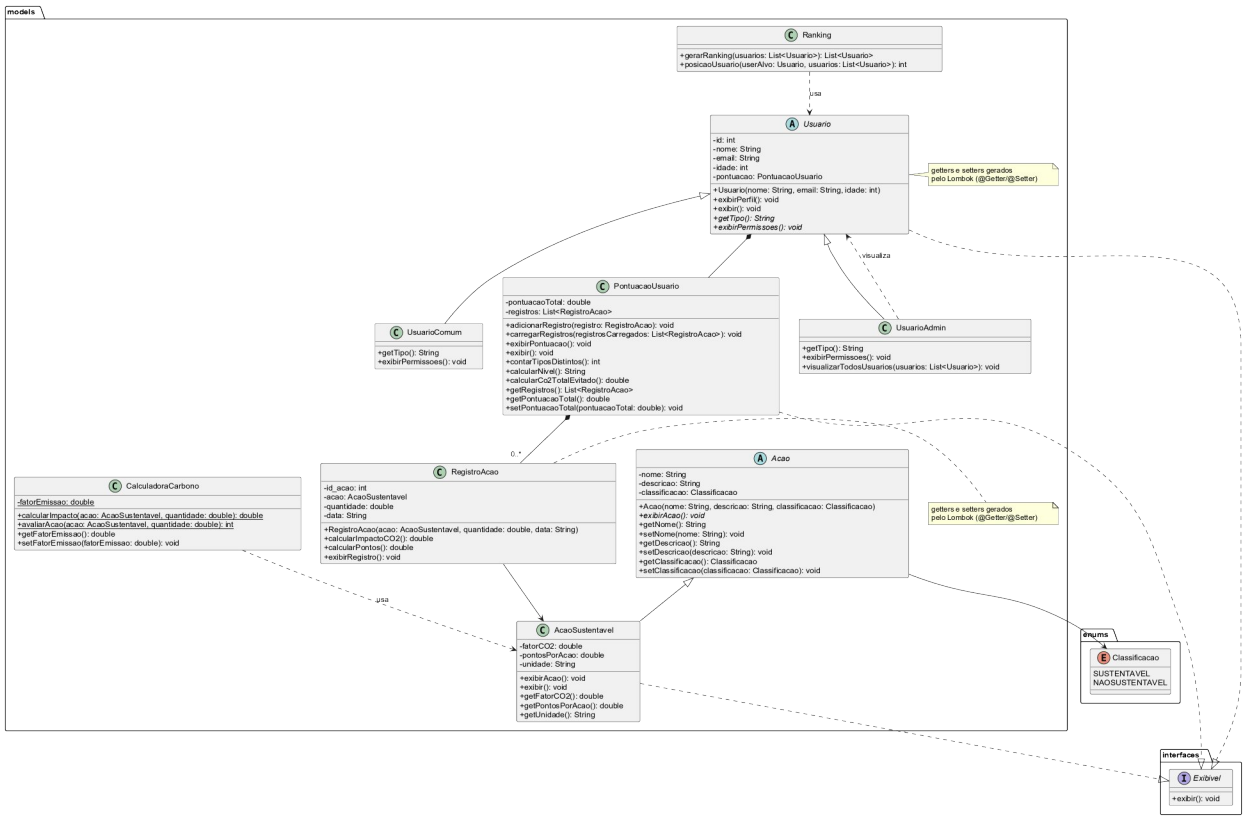
```
CREATE TABLE TBL_USUARIO (
    ID_USUARIO              NUMBER(10)      PRIMARY KEY,
    NOME_USUARIO            VARCHAR2(100)   NOT NULL,
    EMAIL_USUARIO           VARCHAR2(100)   NOT NULL,
    IDADE_USUARIO           NUMBER(3),
    PONTUACAOTOTAL_USUARIO NUMBER(10,2)    DEFAULT 0,
    TIPO_USUARIO            VARCHAR2(10)    NOT NULL
);

CREATE TABLE TBL_REGISTRO_ACAO (
    ID_REGISTRO      NUMBER(10)      PRIMARY KEY,
    ID_USUARIO       NUMBER(10)      NOT NULL,
    NOME_ACAO        VARCHAR2(100)   NOT NULL,
    QUANTIDADE       NUMBER(10,2)    NOT NULL,
    DATA_REGISTRO    VARCHAR2(20)    NOT NULL,
    PONTOS_GANHOS    NUMBER(10,2)    NOT NULL,
    CONSTRAINT FK_REGISTRO_USUARIO FOREIGN KEY (ID_USUARIO)
        REFERENCES TBL_USUARIO(ID_USUARIO)
);
```

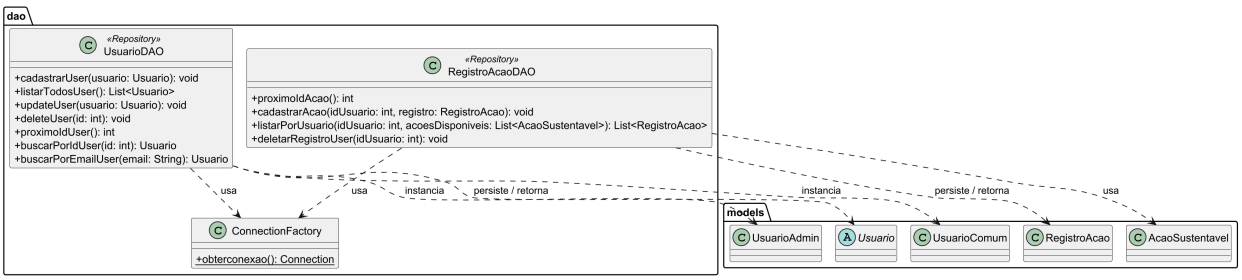
7. Diagrama de Classes

O diagrama de classes foi dividido por camada para manter a legibilidade. Ele mostra atributos, métodos e os relacionamentos de herança, implementação de interface, composição, associação e dependência. (Aumenta o zoom do documento para conseguir ver os diagramas, eles estão em um svg então e possível enxergar com uma alta qualidade.)

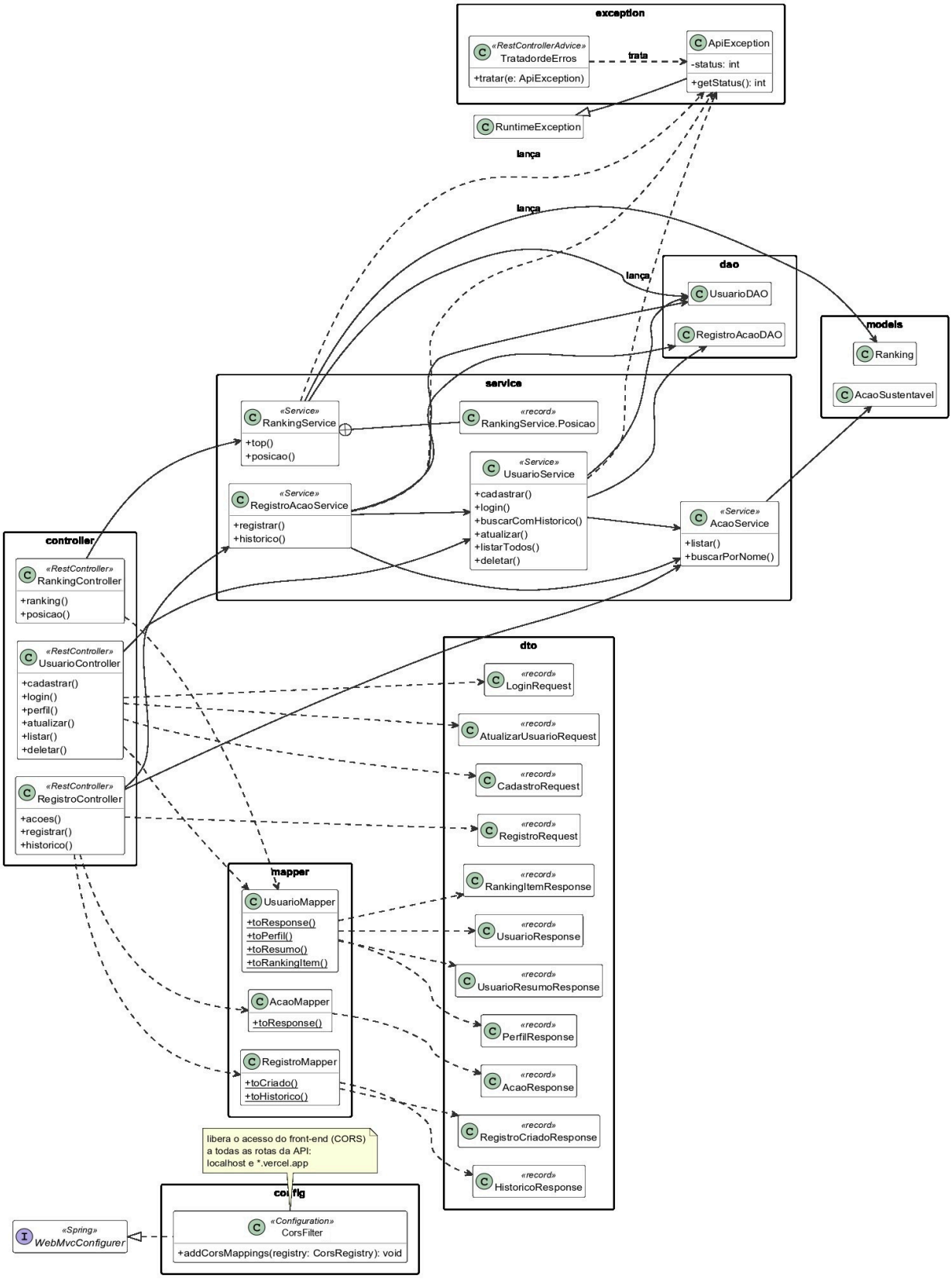
7.1 Pacote models



7.2 Pacote dao



7.3 Pacotes service, controller, dto, mapper e exception



8. Deploy e Acesso

8.1 Links

Repositório no GitHub: https://github.com/RankUp-TDSPJ/RankUp-Java

API publicada (Render): https://rankup-java.onrender.com/acoes

Observação: a API está em um plano gratuito e entra em repouso depois de um tempo sem uso. A primeira requisição após esse período pode demorar alguns instantes.

8.2 Acesso de Administrador

Para cadastrar um usuário como Administrador, informe o código "senha" no campo de código de administrador durante o cadastro. Se o campo ficar em branco ou o código estiver incorreto, o usuário é criado como Comum.

9. Procedimentos para Rodar a Aplicação

9.1 Ferramentas e versões

Item	Versão / observação
Java JDK	21
IDE	IntelliJ IDEA (também abre no Eclipse ou no VS Code como projeto Maven)
Build	Maven, pelo Maven Wrapper incluso no projeto (`mvnw` / `mvnw.cmd`)
Framework	Spring Boot 4.1.1 e Lombok (habilitar *annotation processing* na IDE)
Banco de dados	Oracle da FIAP (`oracle.fiap.com.br:1521/orcl`), driver ojdbc17
Opcional	Docker e Docker Compose

9.2 Como importar o projeto

1. Extraia o arquivo .zip do projeto (ou clone o repositório do GitHub).
2. No IntelliJ IDEA, use **File > Open** e selecione o arquivo `pom.xml` da pasta `Calculador02`; escolha **Open as Project**.
3. Aguarde o Maven baixar as dependências e confirme o JDK 21 em **File > Project Structure**.

9.3 Configuração do banco (variáveis de ambiente)

As credenciais não ficam no código. Antes de executar, defina as variáveis abaixo (no IntelliJ: **Run > Edit Configurations > Environment variables**). Sem elas, as rotas que usam o banco respondem 500.

Variável	Valor
`SPRING_DATASOURCE_URL`	`jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl`
`SPRING_DATASOURCE_USERNAME`	RM do integrante (ex.: `RM000000`)
`SPRING_DATASOURCE_PASSWORD`	senha do Oracle desse RM

As tabelas `TBL_USUARIO` e `TBL_REGISTRO_ACAO` devem existir no schema (DDL na seção 6.1).

9.4 Passo a passo da execução

1. Pela IDE: execute a classe `Calculador02Application` (botão **Run**).
   Ou pelo terminal, na pasta do projeto: `mvnw.cmd spring-boot:run` (Windows) ou `./mvnw spring-boot:run`.
2. A API sobe em `http://localhost:8081`. Teste no navegador: `http://localhost:8081/acoes`.
3. Os demais endpoints podem ser testados no Postman ou Insomnia, conforme a tabela da seção 4.
4. Com Docker (opcional): defina `ORACLE_USER` e `ORACLE_PASSWORD` e rode `docker compose up --build`.

Versão publicada: **https://rankup-java.onrender.com/acoes** (a primeira requisição pode levar cerca de 1 minuto).